ProTalento Integración de React y consumo de datos externos

Universidad Don Bosco

Desarrollo de Aplicaciones Web

Avance 2 Foro de Discusión 2

Equipo A

Cristian Alexander Campos Díaz — CD261570

Luis Alberto Chávez Mendoza — CM250419

Manuel de Jesus Mejía Rivera — MR260122

César Eliseo Ramírez Romero — RR261314

Crystel Daniela Tino Anaya — TA261223

Fecha de entrega: 8 de noviembre de 2026

Alcance de la revisión

Código base revisado: commit `6d8fc88` de `integracion-fase-2`, carpeta `daw-react`. Las pruebas funcionales fueron ejecutadas manualmente por César Ramírez y documentadas mediante confirmaciones y capturas. La compilación de producción finalizó correctamente. Se identificaron dos incidencias pendientes: apertura del CV y reproducción del video desde el perfil administrativo. Las justificaciones de arquitectura corresponden al análisis del código revisado.

1 Introducción y objetivos

ProTalento es una plataforma académica para registrar, evaluar y seleccionar aspirantes a formación técnica de la Agencia Internacional de Desarrollo Local. El proyecto organiza información personal, cursos, convocatorias, documentos y etapas de evaluación. Su primera arquitectura utiliza páginas HTML independientes y JavaScript para controlar formularios, almacenamiento y navegación.

La segunda entrega incorpora React para organizar la interfaz en componentes, administrar cambios de estado y mantener la navegación dentro de una aplicación. Este informe explica la estructura resultante y la integración de información geográfica externa en el registro de aspirantes. También establece las pruebas necesarias para comprobar que la migración conserva las funcionalidades del sistema.

El objetivo general es documentar y verificar la integración de React en ProTalento. Los objetivos específicos son describir la arquitectura anterior y la actual, relacionar los componentes y el estado con las necesidades del sistema, explicar el flujo de la fuente externa y comprobar los módulos mediante pruebas y capturas reales.

2 Descripción de ProTalento y necesidades del proyecto

El aspirante registra sus datos, inicia sesión, consulta su progreso y presenta archivos. El personal administrativo gestiona cursos y convocatorias, consulta indicadores, revisa aspirantes y registra evaluaciones. El código reconoce los roles `usuario`, `administrador` y `superadministrador`.

Estas funciones requieren consistencia entre formularios, listados y estados del expediente. La separación de interfaz y servicios facilita revisar las reglas de negocio y actualizar la presentación sin duplicar toda la lógica. La información territorial requiere selecciones relacionadas para evitar combinaciones de domicilio incoherentes.

3 Investigación de React

React es una biblioteca de JavaScript para construir interfaces mediante componentes. Aunque la guía académica emplea el término framework, su documentación presenta una interfaz compuesta por funciones que devuelven marcado. JSX permite expresar esa estructura junto con la lógica de presentación; las propiedades permiten entregar información a componentes hijos y los eventos permiten responder a las acciones del usuario (React, s. f.-a).

En ProTalento, `Indicadores`, `FiltrosAspirantes`, `TablaAspirantes` y `Graficas` dividen responsabilidades del panel. La composición permite mantener cada sección en un archivo específico y ofrece unidades que pueden reutilizarse con otros datos. La existencia de componentes separados demuestra descomposición; su uso en varias pantallas debe comprobarse antes de afirmar reutilización efectiva entre módulos.

El estado representa datos que modifican lo mostrado en pantalla. React permite mantenerlo en el ancestro común cuando varias partes deben cambiar de forma coordinada y distribuirlo mediante propiedades (React, s. f.-b). El panel aplica este patrón al conservar la búsqueda y los filtros en `PanelControl` y entregar valores y funciones a sus hijos.

`useEffect` sincroniza un componente con sistemas externos y permite definir limpieza al desmontarlo (React, s. f.-c). En el registro se utiliza para solicitar la información territorial. En el panel se registra y elimina un oyente del evento `storage`. La estructura usa además `useMemo` para conservar el resultado del filtrado mientras no cambien sus dependencias. Esto describe la implementación; no demuestra por sí solo una mejora de rendimiento medida.

4 Arquitectura anterior y resultante

La versión anterior distribuye las pantallas entre archivos como `login.html`, `panel-control.html` y `registro_aspirantes.html`. Sus scripts en `js/` atienden eventos y consultan almacenamiento del navegador. Esta estructura y su propósito están descritos en el repositorio del proyecto (Equipo A, s. f.).

La versión React incorpora el siguiente recorrido:

`main.jsx` → `BrowserRouter` → `App.jsx` → `RutaProtegida o página pública` → `página` → `componentes y servicios` → `almacenamiento o fuente externa`.

`main.jsx` monta la aplicación con `createRoot` y `StrictMode`. `App.jsx` declara rutas para inicio, registro, login, portal del aspirante, panel, aspirantes, perfil y convocatorias. `RutaProtegida` consulta la sesión y los roles autorizados; devuelve `Outlet` cuando se permite el acceso y redirige en los demás casos.

La presentación está en `src/paginas/` y `src/componentes/`. Las operaciones de autenticación, consultas, exportaciones y expedientes se ubican en `src/servicios_js/`. Las validaciones y alertas están en `src/utilidades_js/`. Vite configura el entorno de desarrollo y la construcción. Las dependencias declaradas incluyen React, React DOM, React Router, Bootstrap, Chart.js, SweetAlert2, jsPDF y XLSX; las versiones efectivamente instaladas deberán registrarse cuando se complete la instalación.

React cambia la organización de la interfaz, pero la persistencia continúa en el navegador. `localStorage` conserva registros estructurados y sesiones. El servicio del portal

abre `ProTalentoDB`, versión 2, y utiliza los almacenes `curriculums` y `videos` de IndexedDB. No se ha incorporado una base de datos centralizada ni autenticación de servidor.

5 Componentes y responsabilidades

Elemento	Responsabilidad observada
`App`	Declarar rutas y agruparlas por permisos
`RutaProtegida`	Consultar sesión y comprobar roles
`PanelControl`	Coordinar datos, filtros, navegación y exportación
`Indicadores`	Presentar métricas a partir de aspirantes
`FiltrosAspirantes`	Permitir selección de curso y estado
`TablaAspirantes`	Presentar filas y comunicar acciones al padre
`Graficas`	Representar datos y comunicar selección de estado
`RegistroAspirantes`	Administrar formulario, domicilio y validación
Servicios JavaScript	Encapsular operaciones sobre datos y archivos

El panel pasa `aspirantes` a los indicadores y las gráficas; a la tabla entrega `aspirantesFiltrados`, `onEvaluar` y `onDescargarPDF`. El hijo comunica la acción mediante una función recibida. Este diseño evita que la tabla decida por sí sola cómo navegar o generar un documento.

6 Manejo de estado y flujo de datos

`PanelControl` mantiene `aspirantes`, `busqueda`, `filtroCurso`, `filtroEstado` y `nombreUsuario` con `useState`. La función `filtrarAspirantes` recibe los registros y criterios; su resultado se calcula dentro de `useMemo`. Cuando cambia un filtro, React vuelve a representar la tabla con la selección correspondiente.

Los indicadores y las gráficas reciben la colección completa; la tabla y las exportaciones reciben la colección filtrada. Esta diferencia debe explicarse en el video para que el total general no se confunda con el total de resultados visibles. El evento `storage` actualiza el panel cuando otra pestaña modifica datos; también existe un botón de actualización manual. El evento no convierte `localStorage` en un almacén global reactivo dentro de la misma pestaña.

`Login` conserva correo, contraseña, error y carga como estado local. `Convocatorias` conserva colecciones y datos de sus formularios. `PortalAspirante` mantiene información del expediente, archivos seleccionados e indicadores de guardado. No se observó uso de Redux ni de un proveedor de Context en el punto de entrada revisado.

7 Integración de la fuente externa

El registro consume el recurso JSON público:

https://juanmedina100.github.io/departamentos-distritos-municipios-el-salvador/departamentos-distritos-municipios-sv.json

Se trata de una fuente externa estática de datos territoriales; no es un servidor propio de ProTalento. Su uso dentro del registro demuestra integración funcional de datos externos. La aceptación de este tipo de recurso como API para la evaluación debe confirmarse con el docente si existe alguna restricción adicional.

El flujo observado es el siguiente: al montar `RegistroAspirantes`, un efecto llama a `obtenerDomicilio`; el servicio ejecuta `fetch`, comprueba `respuesta.ok`, convierte la respuesta a JSON y verifica que `datos.departamentos` sea un arreglo. El componente almacena el resultado y habilita la selección territorial.

Al cambiar departamento, se reinician municipio y distrito. Al cambiar municipio, se reinicia distrito. Así se impide conservar una selección que pertenecía al departamento anterior. El domicilio seleccionado se incorpora al objeto del aspirante al enviar el formulario.

La comprobación de estado HTTP es necesaria porque `fetch` no rechaza automáticamente una respuesta HTTP de error. La lectura JSON también puede fallar y debe tratarse como parte del proceso de carga (MDN contributors, s. f.).

El componente utiliza `cargandoDomicilio` y `errorDomicilio`. Su bloque `catch` registra el fallo y activa la señal de error; `finally` termina la carga. Una bandera `componenteActivo` evita actualizar el estado después del desmontaje. Esa bandera no cancela la solicitud de red. No se observó cancelación mediante `AbortController` ni un mecanismo explícito de reintento.

La validación de respuesta comprueba la colección superior, pero no valida exhaustivamente cada municipio y distrito. Las pruebas deben incluir fallo de red, estado HTTP no exitoso y datos con estructura inválida.

8 Migración y decisiones técnicas

La migración incorpora páginas JSX y reemplaza la navegación entre documentos por rutas declaradas en React Router. Mantiene servicios JavaScript para conservar reglas del prototipo y reducir cambios simultáneos. La combinación permite revisar por separado el comportamiento y la presentación.

El estado se mantiene cerca de las pantallas que lo utilizan. En el panel, centralizar filtros en el padre permite coordinar tabla, gráficas y acciones de exportación. Separar los componentes del panel facilita su mantenimiento y permite probar entradas y acciones con responsabilidades más limitadas.

Se conservan `localStorage` e IndexedDB para continuar el prototipo sin un servidor de aplicación. Esta continuidad debe comprobarse mediante pruebas de regresión: las diferencias de origen, puerto o navegador pueden hacer que la versión React no vea datos guardados en la versión HTML. No debe afirmarse que los datos anteriores se migran automáticamente entre orígenes.

La rama de trabajo local se sincronizó mediante avance directo con `integracion-fase-2`. No se publicaron cambios remotos. Para la entrega deben fijarse la rama y el commit que efectivamente se demostrarán.

9 Pruebas y evidencias

La matriz adjunta define pruebas de navegación, roles, registro, API, cursos, convocatorias, filtros, exportaciones, evaluación y archivos. Un caso solo se marcará como aprobado después de ejecutarlo y registrar el resultado observado. La revisión del código no sustituye una prueba funcional.

Se ejecutaron 12 comprobaciones de servicios y todas aprobaron. Cubren fechas, documentos, contraseñas, normalización, creación no duplicada de cuentas iniciales, acceso y cierre de sesión, roles y tratamiento de respuestas geográficas. Se utilizó almacenamiento en memoria y red simulada. Estos resultados no verifican las pantallas, IndexedDB ni la disponibilidad real de la fuente externa. El detalle se incluye en el archivo de resultados adjunto.

La compilación de producción mediante `npm.cmd run build` finalizó correctamente con Vite 8.3.6 y 254 módulos transformados. Se generó la carpeta `dist`. El archivo JavaScript principal reportó 1304,60 kB y 415,78 kB comprimido. Vite emitió una advertencia por fragmentos superiores a 500 kB; no impidió la construcción. Dividir la carga por rutas constituye una mejora de optimización pendiente.

En las pruebas manuales se confirmó el registro y consulta de aspirantes, la búsqueda, las exportaciones CSV y Excel, la ficha PDF, la persistencia de calificaciones, observaciones y fecha de evaluación, los filtros del panel y la edición persistente de una convocatoria. El calendario deshabilitó fechas de cierre anteriores al inicio; esto acredita la restricción del selector, no una prueba exhaustiva de entradas manipuladas.

La fuente geográfica cargó departamentos, municipios y distritos. Al cambiar departamento se limpiaron los campos dependientes. Al bloquear únicamente la petición JSON apareció un aviso y se deshabilitaron los campos; al desbloquearla y recargar se recuperó la carga. La prueba inicial con todo el navegador sin conexión no se contabilizó, porque impidió cargar la propia aplicación.

El portal registró CV y video como enviados. La persistencia del estado del CV fue confirmada tras recargar. Sin embargo, el perfil administrativo mostró avisos de archivo registrado sin ruta para abrirlo, tanto para CV como para video. Se consideran dos incidencias funcionales: la confirmación de entrega no demuestra que el administrador pueda recuperar los archivos. La hipótesis técnica es una integración incompleta entre el perfil y el almacenamiento de archivos; debe verificarse antes de corregir.

El acceso a PanelControl sin sesión se comprobó en una ventana InPrivate, que redirigió a Login. La restricción de rol se probó en la ventana habitual con el aspirante, que fue redirigido desde Convocatorias a su portal. No se interpretó como contraseña incorrecta la ausencia del usuario en InPrivate, cuyo almacenamiento está separado. El cierre de sesión de la ventana habitual requiere una comprobación específica adicional.

10 Limitaciones y mejoras futuras

La persistencia es local al navegador. Las sesiones y contraseñas del prototipo se conservan en almacenamiento cliente y las rutas protegidas representan control de interfaz, no autorización en un servidor. Para un servicio real se requiere una arquitectura de autenticación y persistencia centralizada.

La dependencia de la fuente geográfica puede impedir completar correctamente el domicilio cuando no hay conexión. Se propone incorporar reintento, cancelación, validación más detallada y una política de disponibilidad del catálogo. Estas funciones se plantean como mejoras y no como características ya implementadas.

Se requiere corregir y volver a probar la consulta administrativa de CV y video. También quedan pendientes las pruebas exhaustivas de tamaño y formato de archivos, el flujo completo de etapas y el cierre de sesión. El panel presenta el estado Aprobado como Seleccionado aun cuando el expediente muestra etapa 3; el equipo debe validar si esa equivalencia responde al proceso previsto. El video debe mostrar la versión efectivamente probada y declarar las limitaciones vigentes.

11 Conclusiones

El código revisado muestra una transición desde páginas HTML independientes hacia una aplicación React con rutas, componentes y servicios. El panel ofrece un ejemplo concreto de coordinación mediante estado y propiedades. El registro integra información territorial externa dentro de un formulario funcionalmente relacionado con ProTalento.

Las pruebas manuales y la compilación respaldan las funciones comprobadas de registro, evaluación, exportación y carga territorial. El manejo del fallo geográfico y su recuperación fueron verificados. La entrega de archivos presenta dos limitaciones en su consulta administrativa, por lo que el flujo documental todavía no puede considerarse completo.

12 Referencias

Equipo A. (s. f.). *Desafio1_UDB_DAW* [Repositorio de código]. GitHub. https://github.com/cristiancdiaz10-del/Desafio1_UDB_DAW

MDN contributors. (s. f.). *Using the Fetch API*. MDN Web Docs. https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API/Using_Fetch

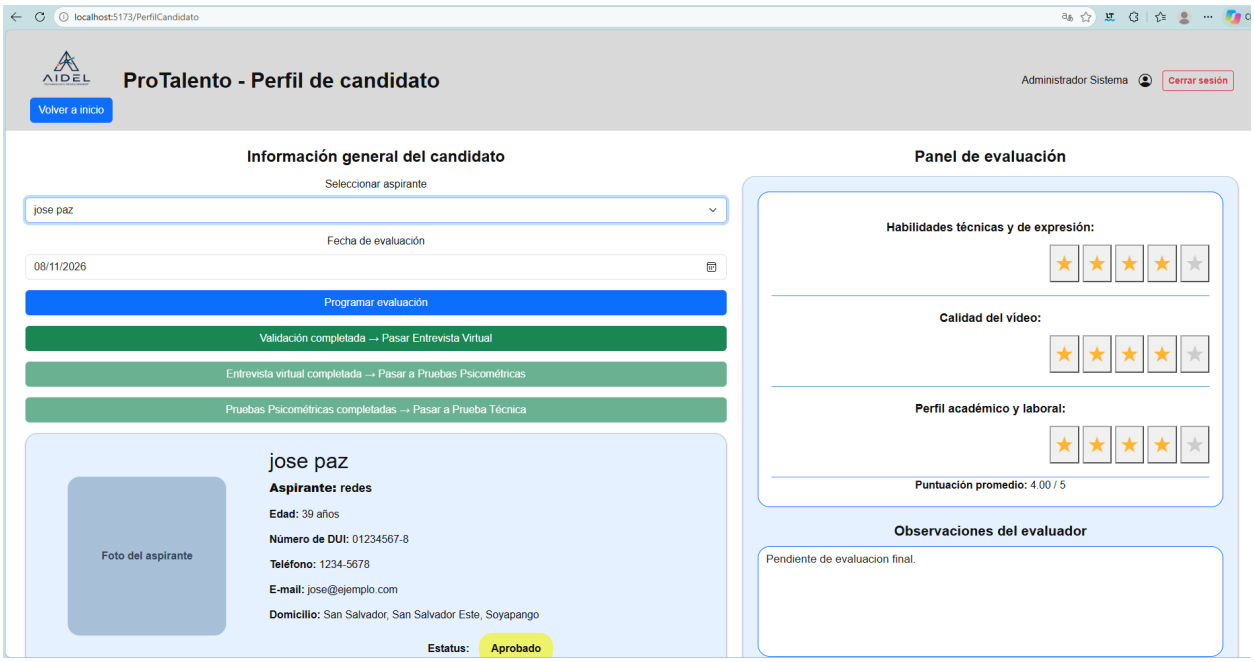
React. (s. f.-a). *Quick start*. https://react.dev/learn

React. (s. f.-b). *Sharing state between components*. https://react.dev/learn/sharing-state-between-components

React. (s. f.-c). *useEffect*. https://react.dev/reference/react/useEffect

Anexo Figura 1

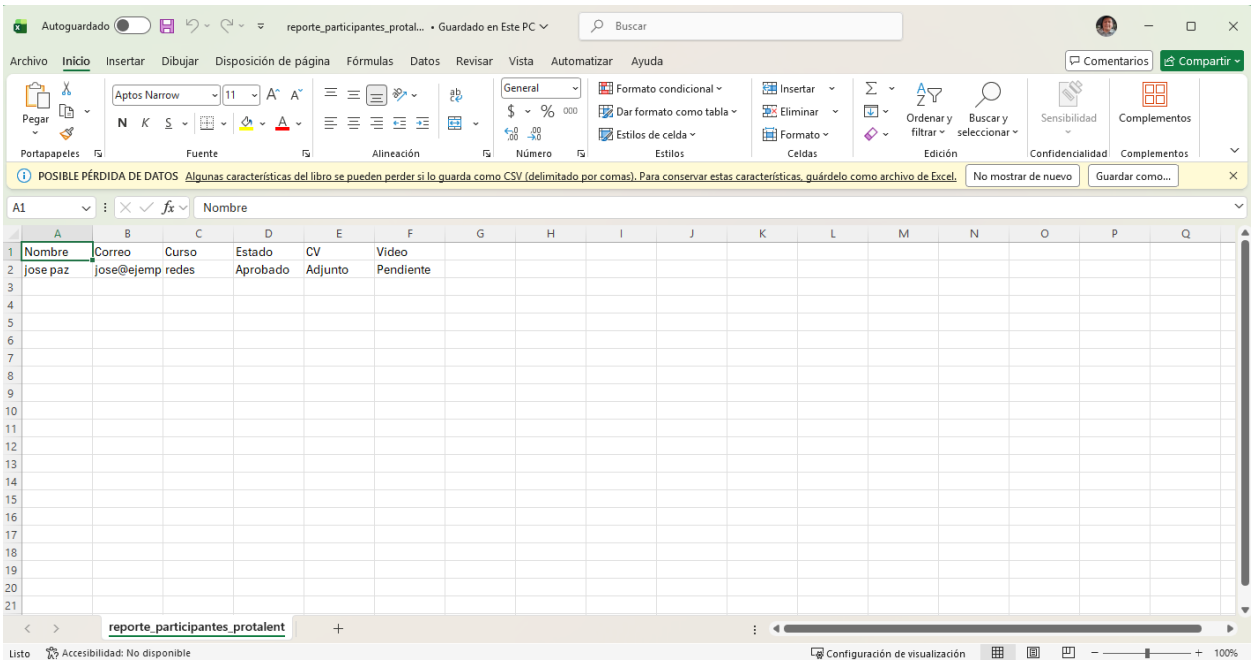
Evaluación y programación



Nota. La captura muestra fecha, calificaciones, promedio y observaciones; su persistencia tras recargar fue confirmada por el usuario. Fuente: captura de la prueba manual aportada por César Ramírez.

Anexo Figura 2

Reporte CSV abierto en Excel

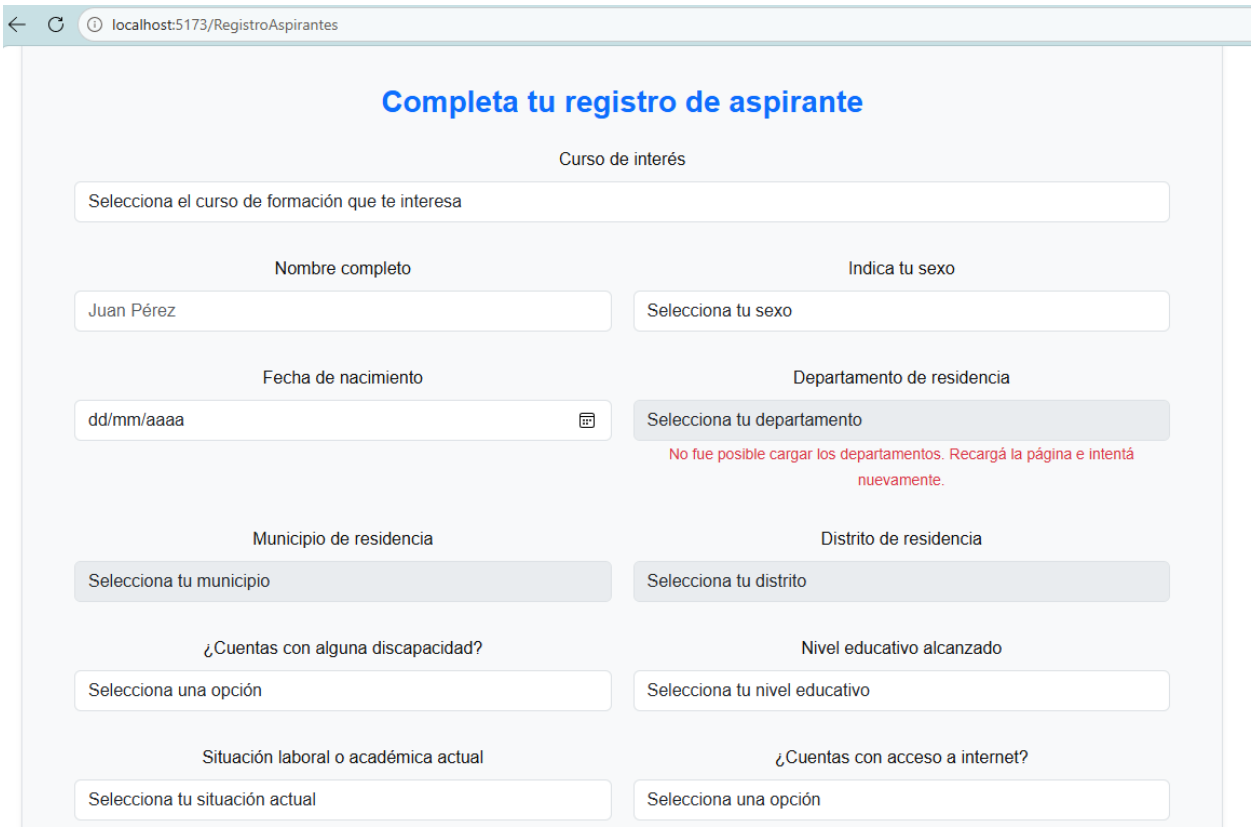


Nota. Las columnas muestran el aspirante, curso, estado y situación documental.

Fuente: captura de la prueba manual aportada por César Ramírez.

Anexo Figura 3

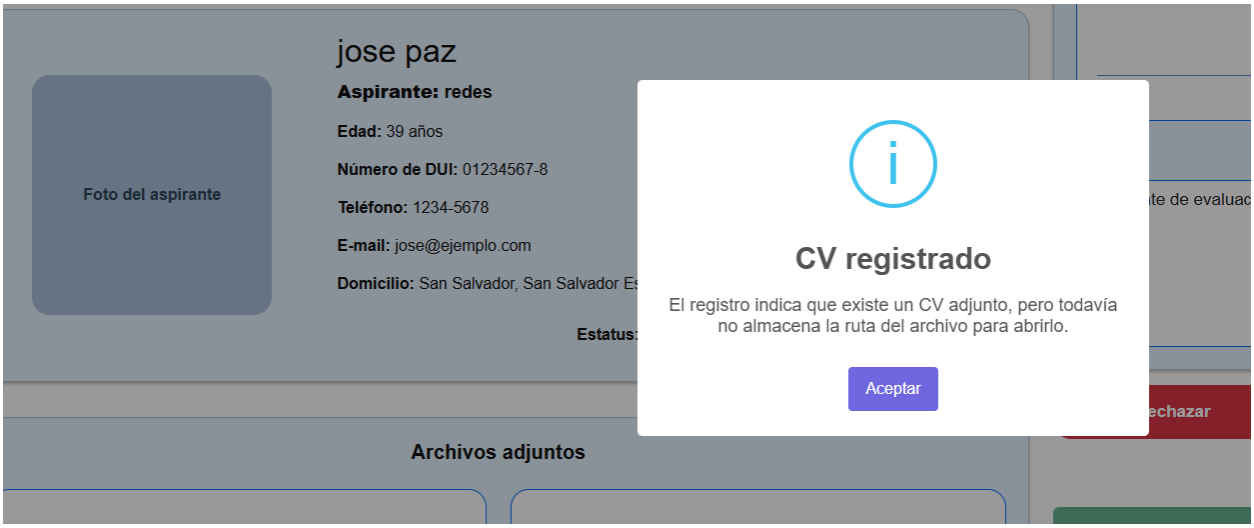
Manejo del error de la fuente geográfica



Nota. El formulario permanece visible, informa la carga fallida y deshabilita los campos geográficos. Fuente: captura de la prueba manual aportada por César Ramírez.

Anexo Figura 4

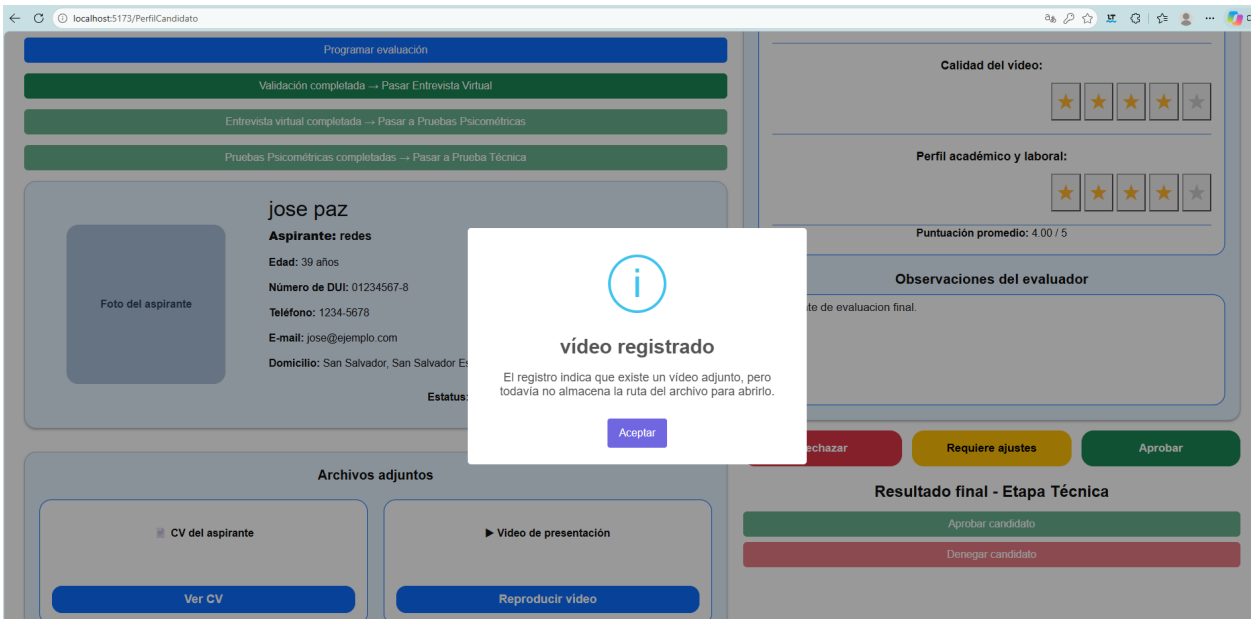
Incidencia al abrir el CV



Nota. El perfil reconoce el adjunto pero no abre el archivo. Fuente: captura de la prueba manual aportada por César Ramírez.

Anexo Figura 5

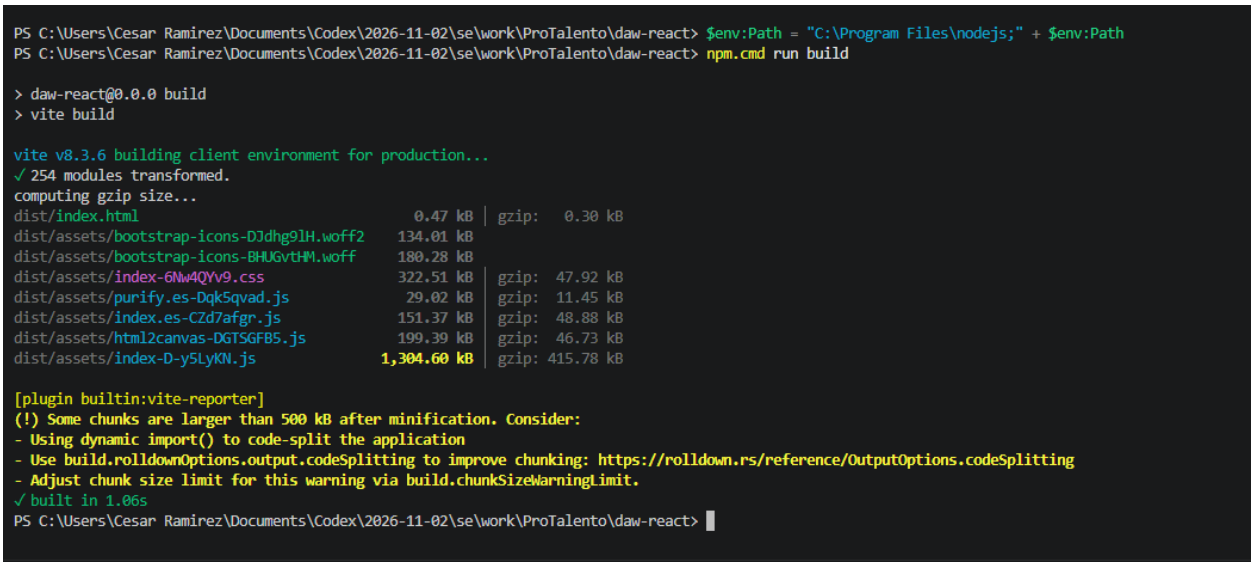
Incidencia al reproducir el video



Nota. El perfil reconoce el video pero no permite reproducirlo. Fuente: captura de la prueba manual aportada por César Ramírez.

Anexo Figura 6

Compilación de producción



Nota. La construcción finaliza correctamente y reporta una advertencia de tamaño.

Fuente: captura de la prueba manual aportada por César Ramírez.